ASSIGNMENT-20.3

Name: Chirra Jyothika

Ht.no:2303A51280

Batch:05

Task 1 – Password Strength & Validation Check

Task:

Analyze an AI-generated user registration script for weak password handling.

Instructions:

• Prompt AI to generate a Python user registration program

• Check whether:

o Password length is validated

o Weak passwords are allowed

• Suggest secure improvements such as:

o Minimum length checks

o Use of regular expressions

o Rejecting common passwords

Expected Output:

A secure registration script with strong password validation rules

Prompt:

J Generate a Python user registration program that accepts a username and password. Then analyze whether the password validation is secure. Improve the script by enforcing strong password rules such as minimum length, use of uppercase and lowercase letters, digits, special characters, and rejection of common passwords using regular expressions.

Code:

```python
import re

def is_strong_password(password):
    # Minimum 8 characters
    if len(password) < 8:
        return False, "Password must be at least 8 characters long"

    # Regex checks
    if not re.search(r"[A-Z]", password):
        return False, "Must include at least one uppercase letter"
    if not re.search(r"[a-z]", password):
        return False, "Must include at least one lowercase letter"
    if not re.search(r"\d", password):
        return False, "Must include at least one number"
    if not re.search(r"[!@#$%^&*(),.?\":{}|<>]", password):
        return False, "Must include at least one special character"

    # Common passwords blacklist
    common_passwords = ["123456", "password", "12345678", "qwerty"]
    if password.lower() in common_passwords:
        return False, "Password is too common"

    return True, "Strong password"


# 🔹 Registration
username = input("Enter username: ")
password = input("Enter password: ")

valid, message = is_strong_password(password)

if valid:
    print("✅ Registration successful")
else:
    print("❌ Error:", message)
```

Output:

```
Enter username: jyothika
Enter password: 12345678
❌ Error: Must include at least one uppercase letter
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/maddi/AppDa
a/Local/Microsoft/WindowsApps/python3.13.exe "c:/Users/maddi/OneDrive/Desktop
Enter username: jyothika
Enter password: 12345678
❌ Error: Must include at least one uppercase letter
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/maddi/AppDa
Enter username: jyothika
Enter password: 12345678
❌ Error: Must include at least one uppercase letter
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/maddi/AppDa
Enter password: 12345678
❌ Error: Must include at least one uppercase letter
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/maddi/AppDa
❌ Error: Must include at least one uppercase letter
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/maddi/AppDa
I ASSISTED CODING/20.4(1).py"
Enter username: Jyothika
Enter password: Jyothika@123
✅ Registration successful
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING>
```

Explanation:

The improved script:

- Prevents weak passwords
- Enforces strong security rules
- Uses regex for validation
- Provides clear error messages

Task 2 – Insecure File Upload Handling

Task:

Evaluate an AI-generated file upload program for security risks.

Instructions:

• Prompt AI to generate a Python or Flask-based file upload

script

• Identify vulnerabilities such as:

o No file type validation

o Unrestricted file size

• Implement secure measures:

o File extension validation

o Size limits

o Safe file storage paths

Expected Output:

A secure file upload implementation that prevents malicious uploads.

Prompt:

```
Generate a Python Flask application that allows users to upload files. Then analyze the code
for security vulnerabilities such as missing file type validation and unrestricted file size.
Improve the implementation by adding file extension validation, size limits, and secure file
storage practices to prevent malicious uploads.
```

Code:

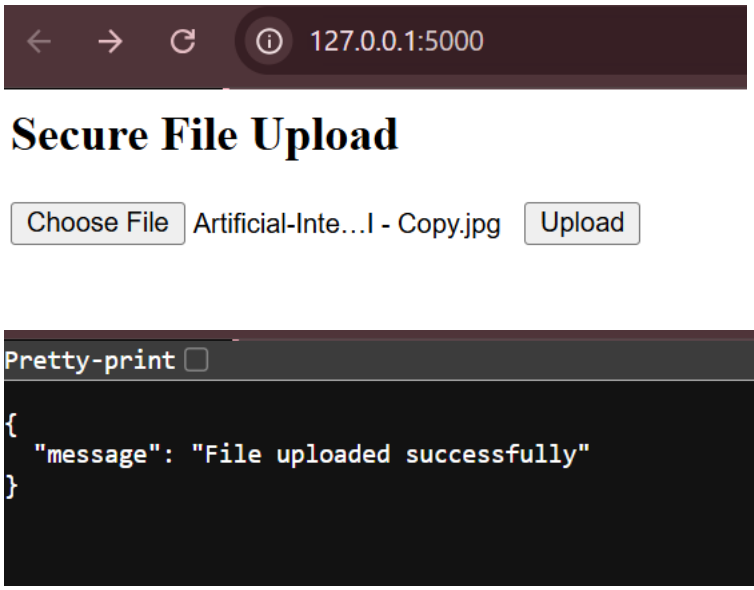
20.4(2).py > upload_file

```python
from flask import Flask, request, jsonify
import os
from werkzeug.utils import secure_filename

app = Flask(__name__)

# ◆ Configuration
UPLOAD_FOLDER = "uploads"
ALLOWED_EXTENSIONS = {"png", "jpg", "jpeg", "pdf"}
MAX_FILE_SIZE = 2 * 1024 * 1024  # 2MB

app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
app.config['MAX_CONTENT_LENGTH'] = MAX_FILE_SIZE

# Create uploads folder if not exists
os.makedirs(UPLOAD_FOLDER, exist_ok=True)


# ◆ Check allowed file types
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS


# ◆ Home Page (Fixes 404 issue)
@app.route('/')
def home():
    return '''
    <h2>Secure File Upload</h2>
    <form method="POST" action="/upload" enctype="multipart/form-data">
        <input type="file" name="file" required>
        <input type="submit" value="Upload">
    </form>
    '''
```

```python
# ◆ Upload Route
@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return jsonify({"error": "No file provided"}), 400

    file = request.files['file']

    if file.filename == '':
        return jsonify({"error": "No selected file"}), 400

    if not allowed_file(file.filename):
        return jsonify({"error": "Invalid file type"}), 400

    filename = secure_filename(file.filename)
    file_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)

    try:
        file.save(file_path)
        return jsonify({"message": "File uploaded successfully"}), 200
    except Exception:
        return jsonify({"error": "File upload failed"}), 500


# ◆ Run App
if __name__ == '__main__':
    app.run(debug=True)
```

Output:

```
* Serving Flask app '20.4(2)'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 139-483-840
127.0.0.1 - - [01/Apr/2026 22:10:28] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Apr/2026 22:10:38] "POST /upload HTTP/1.1" 413 -
127.0.0.1 - - [01/Apr/2026 22:11:54] "POST /upload HTTP/1.1" 200 -
```



Explanation:

The improved implementation:

- Prevents malicious uploads
- Restricts file size and type
- Secures file storage
- Handles errors properly

Task 3 – Real-Time Application: API Security Review

Scenario:

An AI-generated API endpoint is used in a web application.

Instructions:

• Review the API code for:

o Missing authentication

o Insecure HTTP methods

o Lack of rate limiting

• Prompt AI to suggest security improvements such as:

o Token-based authentication

o Input validation

o Rate limiting

Expected Output:

A security-reviewed API endpoint with documented vulnerabilities and fixes.

Prompt:

Provide a Flask-based API endpoint and analyze it for security vulnerabilities such as missing authentication, insecure HTTP methods, and lack of rate limiting. Then improve the API by adding token-based authentication, input validation, and rate limiting to ensure secure access and prevent misuse.

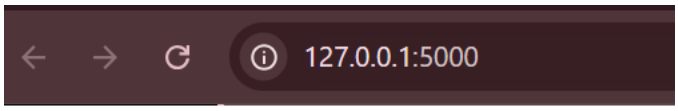
Code:

```python
from flask import Flask, request, jsonify
from functools import wraps
import time

app = Flask(__name__)

# 🔐 Token
VALID_TOKEN = "secure123"

# 🔐 Rate limiting config
request_log = {}
RATE_LIMIT = 5
TIME_WINDOW = 60


# 🔹 Home Route (fixes 404)
@app.route('/')
def home():
    return "API is running! Use /data?user=John"


# 🔹 Token Authentication
def token_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        token = request.headers.get('Authorization')
        if token != VALID_TOKEN:
            return jsonify({"error": "Unauthorized access"}), 401
        return f(*args, **kwargs)
    return decorated


# 🔹 Rate Limiting
def is_rate_limited(ip):
    current_time = time.time()
    request_log.setdefault(ip, [])

    # remove old requests
    request_log[ip] = [t for t in request_log[ip] if current_time - t < TIME_WINDOW]

    if len(request_log[ip]) >= RATE_LIMIT:
        return True

    request_log[ip].append(current_time)
    return False
```

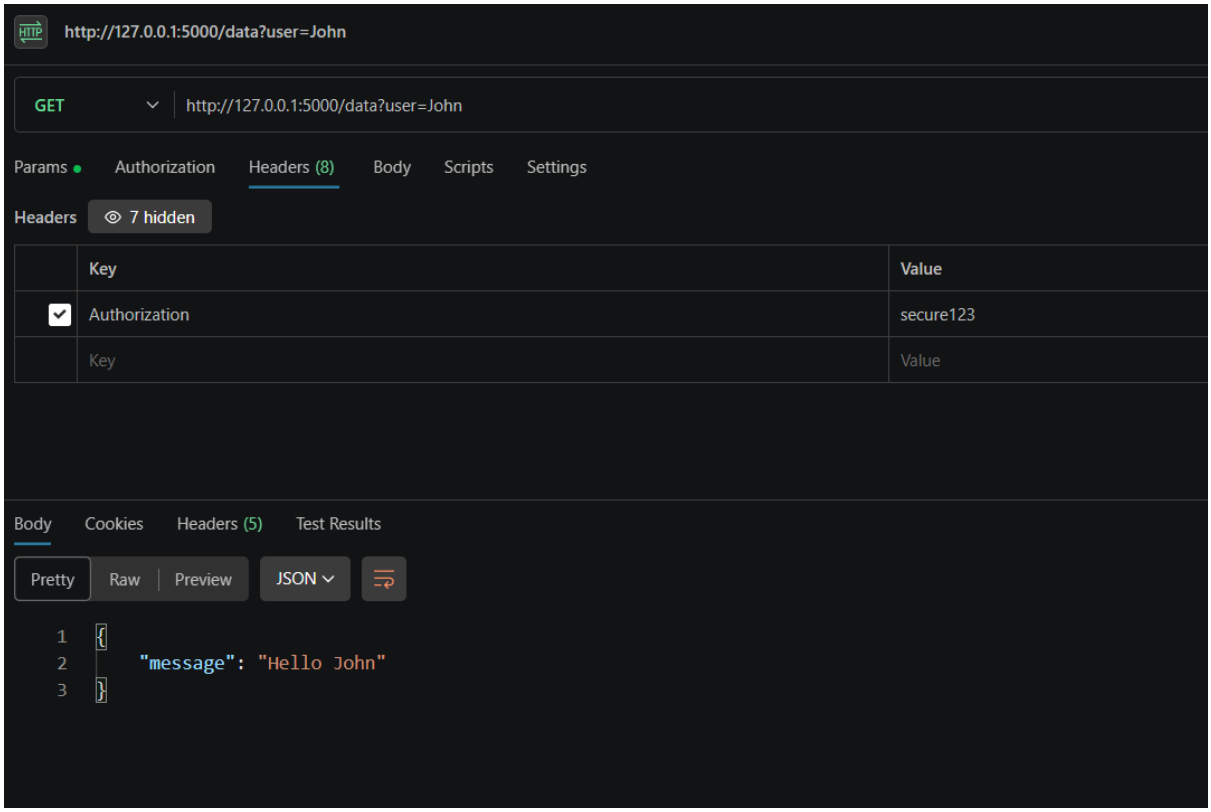
```python
# ◆ Rate Limiting
def is_rate_limited(ip):
    current_time = time.time()
    request_log.setdefault(ip, [])

    # remove old requests
    request_log[ip] = [t for t in request_log[ip] if current_time - t < TIME_WINDOW]

    if len(request_log[ip]) >= RATE_LIMIT:
        return True

    request_log[ip].append(current_time)
    return False


# ◆ Secure API Endpoint
@app.route('/data', methods=['GET'])
@token_required
def get_data():
    ip = request.remote_addr

    if is_rate_limited(ip):
        return jsonify({"error": "Too many requests"}), 429

    user = request.args.get('user', '').strip()

    # ◆ Input validation
    if not user.isalpha():
        return jsonify({"error": "Invalid input"}), 400

    return jsonify({"message": f"Hello {user}"})


# ◆ Run Server
if __name__ == '__main__':
    app.run(debug=True)
```

Output:

```
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 139-483-840
127.0.0.1 - - [01/Apr/2026 22:25:07] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [01/Apr/2026 22:25:24] "GET /data?user=123 HTTP/1.1" 401 -
127.0.0.1 - - [01/Apr/2026 22:25:42] "GET /data?user=John HTTP/1.1" 401 -
127.0.0.1 - - [01/Apr/2026 22:25:44] "GET /data?user=John HTTP/1.1" 401 -
```



API is running! Use /data?user=John



Explanation:

The improved API:

- Secures access using tokens
- Prevents abuse with rate limiting
- Validates user input
- Restricts HTTP methods

Task 4 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated HTML form with JavaScript for XSS

vulnerabilities.

Instructions:

• Ask AI to generate a feedback form with JavaScript-based output.

• Test whether untrusted inputs are directly rendered without escaping.

• Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

- A secure form that prevents XSS attacks.

Prompt:

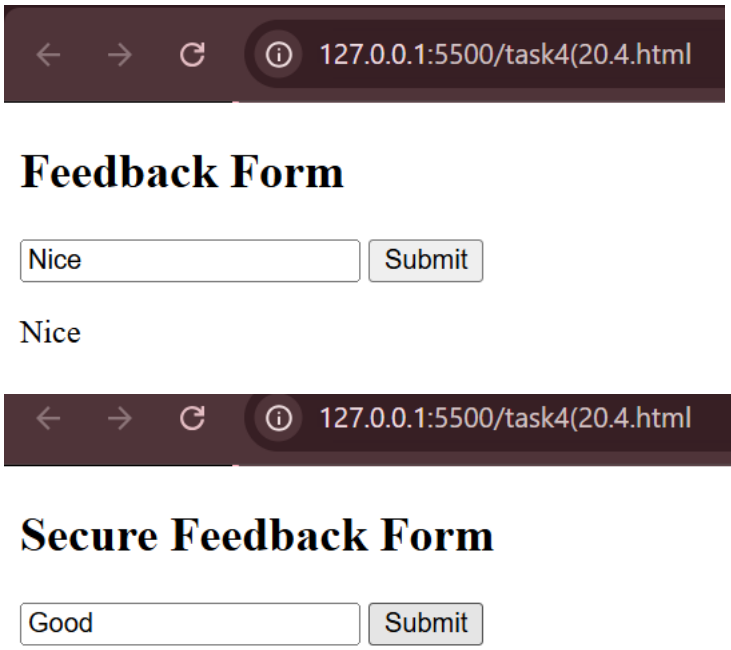
Generate an HTML feedback form with JavaScript that displays user input on the page. Then analyze whether the application is vulnerable to Cross-Site Scripting (XSS) attacks by checking if untrusted input is rendered without sanitization. Improve the code by implementing proper escaping of HTML entities and adding security measures such as Content Security Policy (CSP).

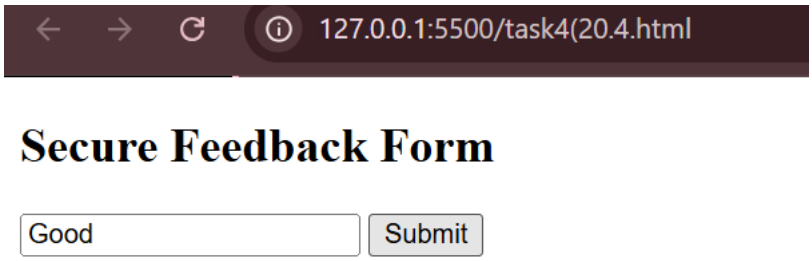
Code:

```html
<!DOCTYPE html>
<html>
<head>
    <title>Feedback Form</title>
</head>
<body>

<h2>Feedback Form</h2>

<input type="text" id="feedback" placeholder="Enter feedback">
<button onclick="submitFeedback()">Submit</button>

<p id="output"></p>

<script>
function submitFeedback() {
    let userInput = document.getElementById("feedback").value;
    document.getElementById("output").innerHTML = userInput;
}
</script>

</body>
</html>
```

```html
<!DOCTYPE html>
<html>
<head>
    <title>Secure Feedback Form</title>

    <!-- ◆ Content Security Policy -->
    <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self';">
</head>
<body>

<h2>Secure Feedback Form</h2>

<input type="text" id="feedback" placeholder="Enter feedback">
<button onclick="submitFeedback()">Submit</button>

<p id="output"></p>

<script>
function escapeHTML(str) {
    return str.replace(/[&<>"']/g, function(match) {
        return {
            '&': '&amp;',
            '<': '&lt;',
            '>': '&gt;',
            '"': '&quot;',
            "'": '&#039;'
        }[match];
    });
}

function submitFeedback() {
    let userInput = document.getElementById("feedback").value;

    // ◆ Escape user input
    let safeInput = escapeHTML(userInput);

    // ◆ Use textContent instead of innerHTML
    document.getElementById("output").textContent = safeInput;
}
</script>

</body>
</html>
```

Output:





Explanation:

The improved form:

- Prevents XSS attacks
- Escapes user input
- Uses secure DOM methods
- Adds CSP protection

Task 5 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database**.**

Instructions:

• Ask AI to generate a Python script using SQLite/MySQL to fetch user details.

• Identify if the code is vulnerable to SQL injection (e.g., using string concatenation in queries).

• Refactor using parameterized queries (prepared statements).

Expected Output:

• A secure database query script resistant to SQL injection.

Prompt:

Generate a Python script that uses SQLite to fetch user details based on a username input. Then analyze whether the script is vulnerable to SQL injection attacks due to string concatenation in SQL queries. Refactor the code using parameterized queries (prepared statements) to ensure the application is secure.

Code:

Database:

create_db.py > ...

```python
import sqlite3

# Create database
conn = sqlite3.connect("users.db")
cursor = conn.cursor()

# Create table
cursor.execute("""
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    username TEXT,
    email TEXT
)
""")

# Insert sample data
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("john", "john@example.com"))
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("alice", "alice@example.com"))
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("Jyothika", "jyothika@example.com"))
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("bob", "bob@example.com"))
cursor.execute("INSERT INTO users (username, email) VALUES (?, ?)", ("joe", "joe@example.com"))

conn.commit()
conn.close()

print("Database created successfully!")
```

task5(20.5).py > ...

```python
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Enter username: ")

# ❌ Vulnerable query (string concatenation)
query = "SELECT * FROM users WHERE username = '" + username + "'"
cursor.execute(query)

result = cursor.fetchall()

for row in result:
    print(row)

conn.close()
```

Output:

```
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> &
Database created successfully!
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> &
Enter username: jyothika
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> &
Enter username: Jyothika
(5, 'Jyothika', 'jyothika@example.com')
PS C:\Users\maddi\OneDrive\Desktop\AI ASSISTED CODING> &
Enter username: bob
(6, 'bob', 'bob@example.com')
```

Explanation:

The improved script:

- Eliminates SQL injection risk
- Uses prepared statements
- Safely handles user input
- Ensures secure database access